

Neural Networks Remove the Need to Detect Spikes Using Traditional Methods

Nikolay Boiarski

The University of Adelaide

Australia

a1822548@student.adelaide.edu.au

Link: <https://github.cs.adelaide.edu.au/a1822548/Topics-in-Computer-Science/releases/tag/v2>

Abstract

Neural activity is a crucial part of modern neuroscience research which is achieved using spike sorting; a process of grouping neuron spikes to individual neurons based on the shape of the spikes. Most spike sorting methods follow a four-step pipeline: filtering, spike detection, feature extraction and clustering, where different algorithms are used for each step making the process very manual and not 100% accurate. The aim of this research is to develop a neural network that can both make the process more automated and have better accuracy than current methods.

Introduction and motivation

Recording neural activity in the brain is an extremely useful technique in neuroscience research [1]. Observing the activity of neurons provides an understanding of the different mechanisms in the brain given various stimulus [2]. One method to detect neural activity involves placing an electrode next to neurons in the brain to detect the electrical current in the neurons near the electrode [2], however, the issue with this method is that multiple neurons are picked up by the electrodes, some of which can fire because of an unrelated source which will be

picked up by the electrode making it difficult to isolate neurons from other neurons as well as from background noise [1]. Neurons can be separated by their waveform, which depends on factors relating to the neuron and electrode placement, which can be used to detect individual neurons. Spike sorting involves grouping the spikes into clusters based on their shape where the clusters can be used to determine the different neurons [3].

There exist many methods for spike sorting, one of which proprietary software with inbuilt algorithms for different sections of the spike sorting pipeline. An example is the Plexon Offline Sorter, however, the limitation of Plexon is that it uses manual thresholds to detect spikes, after which a clustering algorithm is selected, where most methods require manual data inputs. This issue with this method is it is very difficult to standardise due to operator bias [4]. Another method to detect spikes is the usage of neural networks which can be used for multiple sections of the spike sorting pipeline and even combining steps such as feature extraction and clustering together which makes the process more automatic, reducing the risk of operator bias [7][8][10]. Most research into neural networks focuses on

specific sections of the spike sorting pipeline rather than using neural networks from start to finish which is the area of focus for this research.

Review of Current Literature

There are multiple algorithms and software created for spike sorting however, they all follow one main pipeline seen in figure 1. This pipeline process involves first filtering the data to visualise the spikes, second, detecting the spikes from background noise, third, extracting relevant features and fourth, clustering spikes based on their features.

Filtering

It is very difficult to detect spikes from raw neuron spike data, therefore the data must be filtered so that spikes are easier to detect. Plexon has two methods for filtering data, a low-pass and high-pass filter [5]. A low-pass filter removes frequencies below a certain threshold while a high-pass filter removes frequencies above a certain threshold [6]. There also exists a bandpass filter which utilises both a low-pass and a high-pass filter to get frequencies between a certain range. Typically, a high-pass filter of 300hz and a low-pass filter of 3000hz are used [2].

Spike Detection

The spike detection method Plexon offers is a threshold method, where data is determined to be a spike if it goes above a certain threshold value [5]. This limitations with this method is that if the threshold is set too low, then background noise will create false positive spike detections while a threshold that is too high will create false negative detections [2]. It is possible to find the most optimal value, however that can still create false positives or negatives.

A less common method involves the use of neural networks for spike detection. Melinda Rácz et al 2020 used a convolutional neural network for the entire pipeline of spike sorting [7]. The ground truth data which, obtained using KiloSort, which is an automatic spike sorting algorithm, was used to train their neural network. The issue they found with KiloSort is that while being an automatic algorithm, it requires parameter settings that require a human expert to properly input [7]. The KiloSort data was used to train detection, sorting and prediction algorithms. The detection algorithm was a combination of convolutional neural networks and recurrent neural networks which relied on the cluster quality of the KiloSort data [7]. This process had a 69% recall value (where a higher number means better ability to detect true positives) and had different results compared to KiloSort on datasets used to test the performance of the neural network. Melinda Rácz et al 2020 did not test using other methods to train the neural networks, which is a limitation as the usage of KiloSort to train the network may have caused these results. They also did not compare how their neural network compares to a more commonly used pipeline using a threshold value to detect spikes, extracting

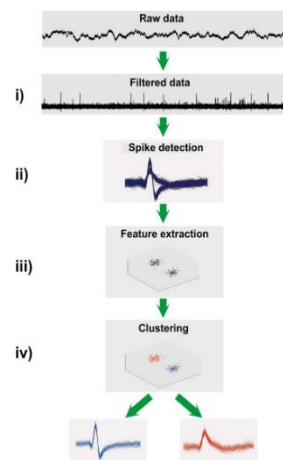


Figure 1: Quiroga, RQ, 2012, *Spike Sorting*, viewed 12 August 2022, <[https://www.cell.com/current-biology/pdf/S0960-9822\(11\)01254-1.pdf](https://www.cell.com/current-biology/pdf/S0960-9822(11)01254-1.pdf)>

features, and feeding that data into a clustering algorithm.

Feature Extraction

Feature extraction is an important step in spike sorting as it involved removing irrelevant features and keeping only those that help classify the spikes [3]. One of the most common methods for feature extraction is principal component analysis [PCA] however, it can occasionally produce implausible results [9].

Mohammadreza Radmanesh et al 2022 used a deep contractive autoencoder to extract features from noisy spike data which then puts its output through a k-means algorithm to cluster the spike data. This data put through a k-means algorithm which labels the output of the autoencoder to train a radial basis function support vector machine classifier (diagram in figure 2) [8]. The results of their research show their method achieving the highest results of sorter accuracy when compared to other sorting algorithms available. Their results also have the fastest running time when sorting data, however, there is significant time taken to train the data, which the model can do while it runs, taking some of the data as training data and sorting the rest of the data.

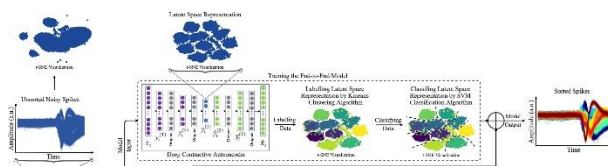


Figure 2: Radmanesh, M, 2022, *Online spike sorting via deep contractive autoencoder*, viewed 12 August 2022, <<https://www.sciencedirect.com/science/article/pii/S089360802200301X>>

Due to this method being created to allow for online spike sorting, it performs these operations on the raw data and does not filter

it and detect spikes beforehand which could increase its capabilities but may increase the running time.

Junsik Eom et al 2021 also used autoencoders to extract features from spikes but only used them as a replacement for other feature extraction methods such as PCA. The method they use involves pre-processing detected spikes into gradient values and then normalising them after which autoencoders extract relevant features [10]. This model produced accurate spike detections, especially for datasets with high levels of noise. However, it used the threshold method for detecting spikes so while their feature extraction may have had good accuracy, the model may have improved performance with a neural network-based spike detection algorithm.

Clustering

Once the important features of spikes have been extracted, they can be grouped together to determine individual neurons [2]. There exist algorithms that can cluster these features together, however the limitation of them is that they require a manual input before they can start. P.M. Horton et al 2007 uses a Kohonen network [better known as a self-organising map] to identify the different clusters [11]. A Kohonen network requires information before it can be run, but P.M. Horton et al 2007 managed to develop software that gets around this issue. The limitation in their research is that they did not compare their results to other clustering algorithms and how their performance changes based on various input data such as noise or number of neurons.

Conclusion

In conclusion, there are multiple algorithms that exist for spike sorting; however, most are manual and require human input during various stages of the sorting process. The usage of deep learning algorithms helps get around this issue by learning what to look for when sorting data. The limitation is that all but Melinda Rácz et al 2020 use neural networks from start to finish in the spike sorting process, which might be the most efficient method. The aim of this research will be to develop software that uses neural networks for most if not all the spike sorting process and how this compares to Plexon software and other algorithms commonly used.

From the literature, it can be seen that there are many sources that feature extracting and sorting spike data, however, there are few that feature ways to filter and detect those spikes. This shows a gap in the research of spike sorting specifically in the process of spike detection. Is there a reason for this? The aim of this project will be to answer this question.

Methodology

To create a pipeline for detecting spikes using neural networks a series of steps must be used to generate the data and process it into a form that the neural network will accept. The next section covers all the steps required to generate and process data and to create a simple neural network that classifies that data.

To perform analysis on spike sorting methods using neural networks, a ground truth of spike data must exist. The ground truth data will have the true spike times, which can be used to teach the model which parts of the data are spikes, and which are not. In this

project, to generate spike data, a MATLAB software called Noisy Spike Generator was used [12]. This software includes multiple parameters that can be altered to generate different types of spike trains such as the number of target neurons, jittered spikes, uncorrelated spikes and much more. Although, most importantly it includes the ability to generate noisy data [12]. It is rare that real spike data will have no noise and, in most cases, will require filtering of the data to somewhat reduce the noise. An example of a noisy sample produced by Noisy Spike Generator can be seen in Figure 3.

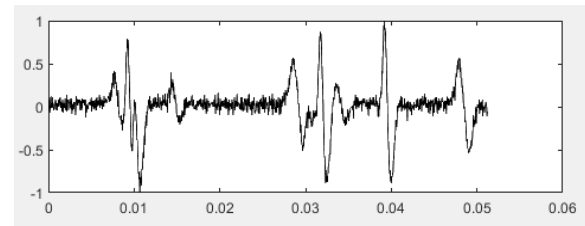


Figure 3: Data Generated Using Noisy Spike Generator

To know the ground truth data of spikes generated by Noisy Spike Generator, a structure is automatically generated that includes a field called 'targettimes' which is the start of each spike. An example of this data is: [0.0075,0.0100,0.0300,0.03750]. Each value represents the point on the x-axis that the spike begins at and can be used to manually check if a spike event is an actual spike or is unrelated, which can then be used to create an appropriate label for the type of event it is.

When dealing with data as a wave, it can be difficult to find libraries that have operations that can be performed on the data both normally and inside a neural network. However, the data generated using Noisy Spike Generator shares characteristics with regular audio in that it has an output channel, which in this case would be mono, and has a sample rate. An inbuilt MATLAB function is

the *'audiowrite'* function which takes in a filename, data and sample rate and produces an audio file [13]. This allows the Noisy Spike Generator data to be transformed into an audio file which allows various audio manipulation function to be used on the spike data making it easier to pre-process before creating the neural network.

In this stage a second audio file was also generated which used the MATLAB inbuilt function *lowpass* which takes in the spike data, passband frequency (which was set to 350hz) and sample rate [17]. This file would be equivalent to the first apart from it having a significant amount of noise reduced.

A neural network that is used to classify data requires an input that has a certain label representing what class it belongs to where a class is one of the possible classifications. In the case of spike data, an input such as that is seen in Figure 3 contains both spike and non-spike events so the model will not be able to classify it. What must be done is the data must be split into individual events which can be achieved using a function from the pydub library called *'split_on_silence'* [14]. This function has a parameter where you can specify how long silence must be before taking it as actual silence. This value can go down to 1 millisecond which was kept constant throughout the whole project as one millisecond when dealing with spikes is not a small amount of time. Another parameter this function has that was kept at a constant 20 dBFS is the *'silence_thresh'* which allows a wave to not be entirely flat to be considered silent which is an incredibly useful tool for noisy spike data which is rarely flat even when passed through a filter such as a low pass filter.

When splitting the audio data, due to the silence occurring at different intervals the files can end up being a different length. Because a neural network requires all parts of the data to be of the same length the data must be padded up to a maximum value. The fixed maximum length for this project was 20ms.

Once the data has been split and made the same length it must be labelled so that it can be used to train a neural network. This process was completed manually where spikes were generated using Noisy Spike Generator, the ground truth values were checked and specific data was chosen based on if it contained one spike or non-spike event.

While the data can be passed to the neural network as an audio file, another technique that can be used is to make it into a Mel Spectrogram which is an 'image' of the audio and then use image classification methods to classify the spike data. But why use Mel Spectrograms?

First regular spectrograms must be examined. Why use a spectrogram? Because most deep learning models rarely take raw audio as an input [15]. However, the limitation of using regular spectrograms is that it takes audio frequencies linearly rather than logarithmically which is an issue a Mel Spectrogram aims to solve [15]. The Mel Spectrogram two methods one of which is the Mel Scale, which scales frequencies in a way that, for example, 100hz to 200hz sounds the same as 1000hz to 1100hz [15]. The other method is the Decibel Scale where the difference between each decibel increases logarithmically [15]. Figure 5 shows an example of the Mel Spectrogram using the Mel and Decibel scales as its axes.

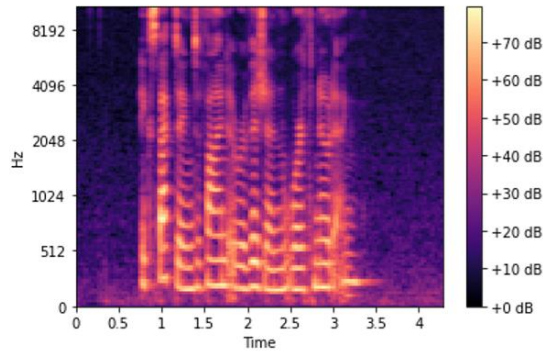


Figure 4 Doshi, K 2021, 'Audio Deep Learning Made Simple (Part 2): Why Mel Spectrograms perform better'. Mel Spectrogram Representation

The last step of the pipeline is an image classifier. The type of network used was a Visual Geometry Group (VGG) which was adapted from the model created by Valerio Velardo – The Sound of AI [16].

Experimental Setup

This section will describe more in detail how the steps in the methodology section were used for this project.

Generating Spikes

The spike/audio generation process was performed in MATLAB while the rest of the code was created in the Google Collab environment due to being able to access their cuda capable machines.

Spikes were generated using Noisy Spike Generator with the function call having parameters: N_Targets, N_Uncorr, Duration, N_Jitter and NoiseSNR. These values were changed with the number of targets ranging from 1-3, the number of uncorrelated ranging from 1-3, jitter ranging from 0-7 and the noise ranging from 10-100 decibels.

Once the spike train was generated, the audiowrite MATLAB function was used to create an audio file. The name of the file ended with .wav to make sure it was

converted into the 'wave' format. The second noise reduced audio file was also created after going through the lowpass filter.

The generated audio files had to be uploaded to the Google Collab environment.

The code for splitting audio data was obtained from source [18] which splits the audio on silence using the parameters mentioned in the methodology section. The resulting code should create various files each containing single spike events.

To create a dataset of spike and non-spike events by hand the spike event output files had to be plotted to know what event each file contained. This data was then referenced with the ground truth data given by Noisy Spike Generator to determine which parts of the data were true spikes and which parts of the data were not. This data was used to create a csv which contained the name of each audio file and a label for the spike event it was. This was then fed into the model adapted from Valerio Velardo – The Sound of AI [16] where the output was an average cross entropy loss value of 0.850.

Results

To begin to understand the results of the project, first the output of the neural network must be looked at. The closer the cross entropy loss value is to 0, the better a model is, which means the model created for this project was rather poor. The reason for this may be to do with the structure of the neural network itself, but this would not cause the value to be this poor. The other reasons this value could be so high is due to the dataset being small and having representations of spikes that are poor due to the audio splitting method having issues performing with high levels of noise.

References

1.

R. J. Vogelstein, K. Murari, P. H. Thakur, C. Diehl, S. Chakrabartty and G. Cauwenberghs, "Spike sorting with support vector machines," The 26th Annual International Conference of the IEEE Engineering in Medicine and Biology Society, 2004, pp. 546-549, doi: 10.1109/IEMBS.2004.1403215.

2.

Hernan Gonzalo Rey, Carlos Pedreira, Rodrigo Quian Quiroga, "Past, present and future of spike sorting techniques," Brain Research Bulletin, Volume 119, Part B, 2015, Pages 106-117, ISSN 0361-9230, <https://doi.org/10.1016/j.brainresbull.2015.04.007>.

[<https://www.sciencedirect.com/science/article/pii/S0361923015000684>]

3.

Quiroga, R.Q., 2012. Spike sorting. *Current Biology*, 22[2], pp.R45-R46.

4.

Alessio P Buccino, Cole L Hurwitz, Samuel Garcia, Jeremy Magland, Joshua H Siegle, Roger Hurwitz, Matthias H Hennig [2020] SpikeInterface, a unified framework for spike sorting eLife 9:e61834

<https://doi.org/10.7554/eLife.61834>

5.

Plexon 2019, "Offline sorter user guide" Plexon, viewed 12 August 2022 <<https://plexon.com/wp-content/uploads/2019/02/Offline-Sorter-v4-User-Guide.pdf>>

6.

Electronics Coach n.d, "Difference between High Pass and Low Pass Filter" Electronics Coach, viewed 12 August 2022 <<https://electronicscoach.com/difference-between-high-pass-and-low-pass-filter.html>>

7.

Melinda Rácz *et al* 2020 "Spike detection and sorting with deep learning" *J. Neural Eng.* **17** 016038

8.

Mohammadreza Radmanesh, Ahmad Asgharian Rezaei, Mahdi Jalili, Alireza Hashemi, Morteza Moazami Goudarzi, "Online spike sorting via deep contractive autoencoder", Neural Networks, 2022, ISSN 0893-6080, <https://doi.org/10.1016/j.neunet.2022.08.001>.

[<https://www.sciencedirect.com/science/article/pii/S089360802200301X>]

9.

P.M. Horton, A.U. Nicol, K.M. Kendrick, J.F. Feng, “Spike sorting based upon machine learning algorithms [SOMA]”, *Journal of Neuroscience Methods*, Volume 160, Issue 1, 2007, Pages 52-68, ISSN 0165-0270, <https://doi.org/10.1016/j.jneumeth.2006.08.013>.

[<https://www.sciencedirect.com/science/article/pii/S0165027006004225>]

10.

Junsik Eom, In Yong Park, Sewon Kim, Hanbyol Jang, Sanggeon Park, Yeowool Huh, Dosik Hwang, “Deep-learned spike representations and sorting via an ensemble of auto-encoders”, *Neural Networks*, Volume 134, 2021, Pages 131-142, ISSN 0893-6080, <https://doi.org/10.1016/j.neunet.2020.11.009>.

[<https://www.sciencedirect.com/science/article/pii/S0893608020303944>]

11.

P.M. Horton, A.U. Nicol, K.M. Kendrick, J.F. Feng, “Spike sorting based upon machine learning algorithms [SOMA]”, *Journal of Neuroscience Methods*, Volume 160, Issue 1, 2007, Pages 52-68, ISSN 0165-0270, <https://doi.org/10.1016/j.jneumeth.2006.08.013>.

[<https://www.sciencedirect.com/science/article/pii/S0165027006004225>]

REFERENCE FROM HERE

12.

Smith, L. S. 2017, *Manual for the Noisy Spike Generator MATLAB Software: version 1.2*, University of Stirling Stirling FK9 4LA, Scotland, UK,
<https://www.cs.stir.ac.uk/~lss/noisyspikes/webV1_1/spikegeneratormanual.pdf>

13.

MathWorks n.d., *audiowrite*, MathWorks, viewed 2 November 2022,
<<https://au.mathworks.com/help/matlab/ref/audiowrite.html>>

14.

Organisation/Author, A Year, Title of website, Company/Sponsor Name, Company/Sponsor Location, viewed Day Month Year, <URL>.

<https://github.com/jiaaro/pydub/blob/master/API.markdown>

15.

Doshi, K 2021, 'Audio Deep LEarning Made Simple (Part 2): Why Mel Spectrograms perform better', Intuitive Audio Deep Learning Series, A Gentle Guide to processing audio in Python. What are Mel Spectrograms and how to generate them, in Plain English, 19 February, viewed 2 November 2022, <<https://towardsdatascience.com/audio-deep-learning-made-simple-part-2-why-mel-spectrograms-perform-better-aad889a93505>>

16.

Velardo, V 2021, *PyTorch for Audio + Music Processing*, online video playlist, 12 July, viewed 2 November 2022, <[PyTorch for Audio + Music Processing](#)>



17.

MathWorks n.d., lowpass, MathWorks, viewed 2 November 2022, <<https://au.mathworks.com/help/signal/ref/lowpass.html>>

18.

Stack Overflow 2017, *Split audio files using silence detection*, Stack Overflow, viewed 2 November 2022, <<https://stackoverflow.com/questions/45526996/split-audio-files-using-silence-detection>>